

Functional Programming Exercises

In these exercises, you will practice writing pure functions and avoiding side effects, using lambda expressions for short inline transformations, applying `map()`, `filter()` and `reduce()` to transform, select and aggregate data, using `functools.partial` to pre-configure a function, and composing these tools into small functional pipelines, including with pandas DataFrames.

EXERCISE 1

Pure Functions & Avoiding Side Effects

Objective

Practice writing a pure function, and refactor an impure function so it no longer mutates its input.

Instructions

Complete `calculate_total(prices, tax_rate)` so it returns the total cost including tax. Then fix `apply_discount()` so it returns a NEW list instead of mutating the list it was given.

```
exercise_1_pure_functions.py

def calculate_total(prices, tax_rate):
    # TODO: return the sum of prices, increased by tax_rate
    # e.g. calculate_total([10, 20], 0.1) -> 33.0
    pass

# TODO: fix apply_discount so it no longer mutates the input list.
# It should return a NEW list of discounted prices instead.
def apply_discount(prices, discount):
    for i in range(len(prices)):
        # side effect: this line mutates prices!
        prices[i] = prices[i] * (1 - discount)
    return prices

if __name__ == "__main__":
    print(calculate_total([10, 20], 0.1))    # Expected: 33.0

    original = [100, 200]
    discounted = apply_discount(original, 0.1)
    print(discounted)    # Expected: [90.0, 180.0]
    print(original)
    # Expected: [100, 200] (unchanged!)
```

EXERCISE 2

Lambda Functions

Objective

Practice writing small lambda functions and using them as sort keys and predicates.

Instructions

Replace each None with a lambda that satisfies the comment above it.

```
exercise_2_lambda_functions.py
students = [("Amy", 82), ("Bo", 67), ("Cy", 91), ("Dee", 74)]

# TODO: a lambda that extracts the score (2nd element) of a student tuple
get_score = None

# TODO: sort students by score, highest first, using get_score as the key
ranked = None

# TODO: a lambda that returns True if a score is a "pass" (>= 70)
is_pass = None

if __name__ == "__main__":
    print(ranked)
    # Expected: [('Cy', 91), ('Amy', 82), ('Dee', 74), ('Bo', 67)]

    print(is_pass(74))    # Expected: True
    print(is_pass(67))    # Expected: False
```

EXERCISE 3

Transforming and Filtering with map() and filter()

Objective

Use map() and filter() to transform and select data without writing a manual for-loop.

Instructions

Complete temperatures_f using map(), and warm_days using filter().

```
exercise_3_map_filter.py

temperatures_c = [0, 21, 33, -5, 40, 15]

# TODO: use map() to convert every Celsius value to Fahrenheit
# formula: F = C * 9/5 + 32
temperatures_f = None

# TODO: use filter() to keep only temperatures above 20 Celsius
warm_days = None

if __name__ == "__main__":
    print(list(temperatures_f))
    # Expected: [32.0, 69.8, 91.4, 23.0, 104.0, 59.0]

    print(list(warm_days))
    # Expected: [21, 33, 40]
```

EXERCISE 4

Aggregating Data with reduce()

Objective

Use `functools.reduce` to fold a sequence of items down to a single aggregated value.

Instructions

Complete `total_price` and `most_expensive` using `reduce()`.

```
exercise_4_reduce.py

from functools import reduce

basket = [
    {"item": "bread", "price": 2.5},
    {"item": "milk", "price": 1.8},
    {"item": "eggs", "price": 3.2},
]

# TODO: use reduce() to compute the total price of everything in the basket
total_price = None

# TODO: use reduce() to find the most expensive item (the dict itself)
most_expensive = None

if __name__ == "__main__":
    print(round(total_price, 2))      # Expected: 7.5
    print(most_expensive["item"])    # Expected: eggs
```

EXERCISE 5

Higher-Order Functions & functools.partial

Objective

Build a higher-order function and use `functools.partial` to pre-configure it.

Instructions

Complete `convert_currency()`, then use `partial()` to build `to_usd()` and `to_gbp()`.

```
exercise_5_partial.py

from functools import partial

def convert_currency(amount, rate):
    # TODO: return amount converted using rate (amount * rate)
    pass

# TODO: use partial() to create to_usd(amount), fixing rate=1.09
to_usd = None

# TODO: use partial() to create to_gbp(amount), fixing rate=0.79
to_gbp = None

if __name__ == "__main__":
    print(round(to_usd(100), 2))    # Expected: 109.0
    print(round(to_gbp(100), 2))    # Expected: 79.0
```

EXERCISE 6

Composing a Functional Pipeline

Objective

Combine map, filter and reduce into a pipeline, then reproduce the same result using pandas.

Instructions

Complete `pipeline_total` using filter/map/reduce, then complete `pandas_total` using a DataFrame and `apply()`.

```
exercise_6_functional_pipeline.py

from functools import reduce
import pandas as pd

orders = [12, -4, 30, 8, -2, 19]

# TODO: build a pipeline using filter, map and reduce that:
#   1. keeps only positive orders
#   2. applies a 5% service charge to each (multiply by 1.05)
#   3. sums the result into a single total
pipeline_total = None

# TODO: repeat the same calculation using a pandas DataFrame:
#   - df already has a column "order" from orders
#   - add a column "charged" using .apply() with a lambda that applies
#     the 5% service charge, but only to positive orders (use 0 otherwise)
#   - sum the "charged" column into pandas_total
df = pd.DataFrame({"order": orders})
pandas_total = None

if __name__ == "__main__":
    print(round(pipeline_total, 2))    # Expected: 72.45
    print(round(pandas_total, 2))     # Expected: 72.45
```